Exercise 4

Add a *Members* column to the `BandAdmin` listing page that contains multiple links, one to each `Musician` in the `Band`. Note that appending to a string makes it unsafe again. Use `mark_safe` to fix that.

To show the members associated with a band, you need to create and display a custom column that contains a link to the `MembersAdmin` page for each associated musician. The new version of `BandAdmin` is shown in the following listing.

Listing E.21 `BandAdmin` class with links to associated musicians

```
# RiffMates/bands/admin.py
...
@admin.register(Band)
class BandAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "show_members")
    search_fields = ("name",)

    def show_members(self, obj):
        members = obj.musicians.all()
        links = []

        url = reverse("admin:bands_musician_changelist")

        for member in members:
            parm = f"?id={member.id}"
            link = format_html(
                '<a href="{}{}">{}</a>', url, parm, member.last_name
            )
            links.append(link)

        return mark_safe(", ".join(links))

    show_members.short_description = "Members"
```

A custom column to display links to musicians

Query all Musician objects associated with the Band being processed.

Each musician results in its own link, which will be stored in this list.

Look up the MusicianAdmin page's URL.

Use format_html() to create a link to each musician's Admin page.

Join the link data into a single string. Processing the strings marks them as unsafe again, so they need to be passed to mark_safe() to ensure the HTML is displayed unescaped by the Django Admin.

E.5 Chapter 6

Chapter 6 introduced users and user management, including how to store data associated with a user and deal with passwords.

Exercise 1

The `@login_required` decorator only checks if a user got authenticated. A more generic view restriction can be performed using the `@user_passes_test` decorator in the same library. It takes one required argument: a reference to a function that takes the `User` object as an argument and returns `True` if the user should be allowed